

Process Modeling Recommender Systems

A Generic Data Model and Its Application to a Smart Glasses-based Modeling Environment

Michael Fellmann · Dirk Metzger · Sven Jannaber ·
Novica Zarvic · Oliver Thomas

Received: 1 March 2017 / Accepted: 4 October 2017 / Published online: 12 January 2018
© Springer Fachmedien Wiesbaden GmbH, part of Springer Nature 2018

Abstract The manual construction of business process models is a time-consuming, error-prone task and presents an obstacle to business agility. To facilitate the construction of such models, several modeling support techniques have been suggested. However, while recommendation systems are widely used, e.g., in e-commerce, these techniques are rarely implemented in process modeling tools. The creation of such systems is a complex task since a large number of requirements and parameters have to be taken into account. In order to improve the situation, the authors have developed a data model that can serve as a backbone for the development of process modeling recommender systems (PMRS). This article outlines the systematic development of this model in a stepwise approach using established requirements and validates it against a data model that has been reverse-engineered from a real-world system. In a last step, the paper illustrates an

exemplary instantiation of the data model in a Smart Glasses-based modeling environment and discusses business process agility issues. The authors expect their contribution to provide a useful starting point for designing the data perspective of process modeling recommendation features that support business agility in process-intensive environments.

Keywords Enterprise process modeling · Business agility · Recommender systems · Requirements · Data model · Smart Glasses

1 Motivation and Relevance

For CEOs and CIOs, it is crucial to be aware of the prevailing trends regarding management issues and information technology (IT). Some recent international studies have shown that “business agility and speed to market” as well as “business process management and business process reengineering” belong to the top five management topics (Luftman et al. 2012, 2013). Such top concerns need to be addressed adequately by the respective leaders in order to keep their enterprises competitive. Digital business models and innovations are developed much faster through the increasing usage of IT and need to be adapted permanently. Hence, quickly changing business processes, e.g., due to frequent reconfigurations in enterprises, need to be managed in an agile way. After two decades of research on business process modeling, the (re)construction of semi-formal process models is still a highly manual task involving substantial human effort. Today, in our fast-moving world, a quicker approach for managing and modeling business processes is required. Regarding the modeling activity, some effort is necessary to create

Accepted after two revisions by the editors of the special issue.

Jun.-Prof. Dr. M. Fellmann (✉)
Institute of Computer Science, University of Rostock, Albert-
Einstein-Str. 22, 18059 Rostock, Germany
e-mail: michael.fellmann@uni-rostock.de

Dr. D. Metzger · S. Jannaber · Dr. N. Zarvic · Prof. Dr.
O. Thomas
Information Management and Information Systems, Osnabrück
University, Katharinenstr. 3, 49074 Osnabrück, Germany
e-mail: dirk.metzger@uni-osnabrueck.de

S. Jannaber
e-mail: sven.jannaber@uni-osnabrueck.de

Dr. N. Zarvic
e-mail: novica.zarvic@uni-osnabrueck.de

Prof. Dr. O. Thomas
e-mail: oliver.thomas@uni-osnabrueck.de

models conforming to specified rules regarding the naming of model elements and the abstraction level of model elements. Concerning the naming of model elements, terminological problems are amongst the main problems when using conceptual (process) models (Sarshar et al. 2006). Moreover, the complexity of today's business processes leads to always increasing efforts and difficulties. Thus, it might not be easy to decide where to start and end a modeling process and on which abstraction level to model (Wilmont et al. 2010; Nielen et al. 2011) since current tools are lacking guidance. These obstacles call for process modeling support features which assist users during process modeling and make suggestions on how to complete a process model currently being edited. Such assistance functions can be implemented following the paradigms and principles of recommender systems. For a general overview of recommender systems, see the comprehensive works from Jannach and Ricci (Jannach 2011; Ricci et al. 2011). Systems presenting suggestions to users are common in programming environments (in terms of auto-completing, e.g., Java code) or e-commerce systems (e.g., amazon.com) and their usefulness has been demonstrated in these domains (Sen et al. 2010; Kuschke and Mäder 2014). Still, they are not common in modeling tools. Since recommender systems “generate meaningful recommendations to a collection of users” (Melville and Sindhvani 2010), development activities towards such systems are promising in order to also apply the support of assistance functions in process modeling tools.

Up to now, some proposals have been made in the area of recommendation-based process modeling leading to prototypical developments (Hornung et al. 2008; Koschmider et al. 2011) such as auto-completion approaches (Wieloch et al. 2011; Kuschke and Mäder 2014; Born et al. 2009; Mazanek et al. 2008) or auto-suggest features (Mazanek and Minas 2009; Clever et al. 2013) or recommendation methods for improving business process modeling (Li et al. 2014). However, these contributions rarely provide an explicit and detailed data model. Thus, engineering such systems has to start from scratch regarding the data perspective. It is quite a difficult task since a large number of requirements and parameters have to be addressed. To improve the situation and to fill this gap, we have systematically developed a requirements-based data model for process modeling recommender systems (PMRS) that can serve as a backbone for the development of modeling recommender systems. The development is based on a requirements catalogue previously derived from a literature analysis as well as from three different empirical studies that also involve business users (Fellmann et al. 2015b). It is a revised and extended version of (Fellmann et al. 2016). In this contribution, we provide a revised and more elaborate description how the integrated data model was constructed. Moreover, we also provide a

critical discussion of the model by contrasting it to a model representing a real-world system. In addition to this, we also critically review the compatibility of our data model with the requirements and data demands originating from established recommender system paradigms. Finally, we show an instantiation of this data model in a real-world application.

The article is structured as follows. In a first step, we describe methodological aspects (Sect. 2). We then systematically construct the data model based on requirements before we present the integrated model (Sect. 3). After critically reviewing our model (Sect. 4), we present an illustration on the basis of an exemplary use scenario (Sect. 5) and finish with a summary and conclusion (Sect. 6).

2 Methodological Considerations

We have developed the model in a systematic, stepwise approach using established requirements. These requirements were elicited from literature (for short: *R-Lit*), from a survey among practitioners (for short: *R-Prac*), from a case study (for short: *R-Case*) and by demonstrating a prototypical system to real users (for short: *R-Prot*) who applied it and commented on their experience. The detailed requirements elicitation was part of our previous work (Fellmann et al. 2015b). Since scientific progress is cumulative, we need to re-introduce these requirements throughout this paper in order to be able to construct our data model. The construction of the data model is done in a step-wise procedure. Thereby, requirements extracted from the sources mentioned before are used to derive partial data models. These models are then merged into a single integrated model representing the superset of the partial models. After the model being constructed, we review our model critically. In doing so, we compare our preliminary integrated model with a data model being reverse-engineered from a real-world system. This critical review leads to refinements of the integrated model. We then also analyze the applicability of our model for different recommender systems paradigms in order to ensure a broad applicability of our model. After revisions have been made, the application of the final integrated model is demonstrated in a real-world system. Our research process is depicted in Fig. 1. Solid arrows represent a transformation relation, dashed arrow means “input for consideration”.

3 Requirement-based Design of the Reference Model

In this section, we systematically derive our data model based on requirements. We denote it as a *reference model* since it meets two important characteristics of such models

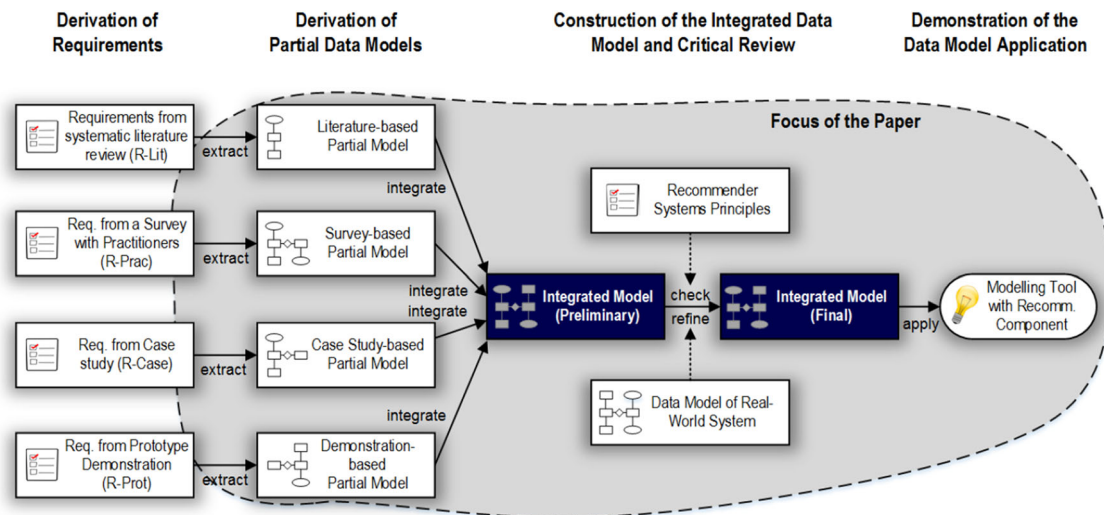


Fig. 1 Overview of the research process

(Thomas 2005, 2006): The model is intended to be generic (that is, not for a specific system) and it is recommended by the authors. Moreover, we use the term *implementation* to denote that a requirement is embodied in the data model, meaning the data model reflects the requirement.

3.1 Implementation of Requirements from Literature (R-Lit)

Requirements from literature have been elicited by reviewing related works found in the databases Science Direct, ISI Web of Knowledge, Springer and EBSCO covering in sum cover approx. 950.000 journals, books and conferences. Requirements have been extracted from 28 pertinent works in a process of a structured literature review following (Webster and Watson 2002) and (vom Brocke et al. 2009). The result of this review and its method is described more detailed in (Fellmann et al. 2015b). Some important works among this set of articles are (Koschmider 2007; Shahzad et al. 2010; Wieloch et al. 2011; Clever et al. 2013; Kuschke and Mäder 2014). Requirements fall into three broad categories: Requirements regarding the content that is recommended as well as in functional and non-functional requirements. Whereas functional requirements represent the (executable) features of a system, non-functional requirements are also referred to as “quality requirements” and represent quality issues of the overall system, i.e., they “specify how well the system must perform its functions” (Lauesen 2002, p. 217). We introduce each requirement briefly and comment on how these requirements are reflected in the data model. The data model in this section (Fig. 2) and in subsequent sections

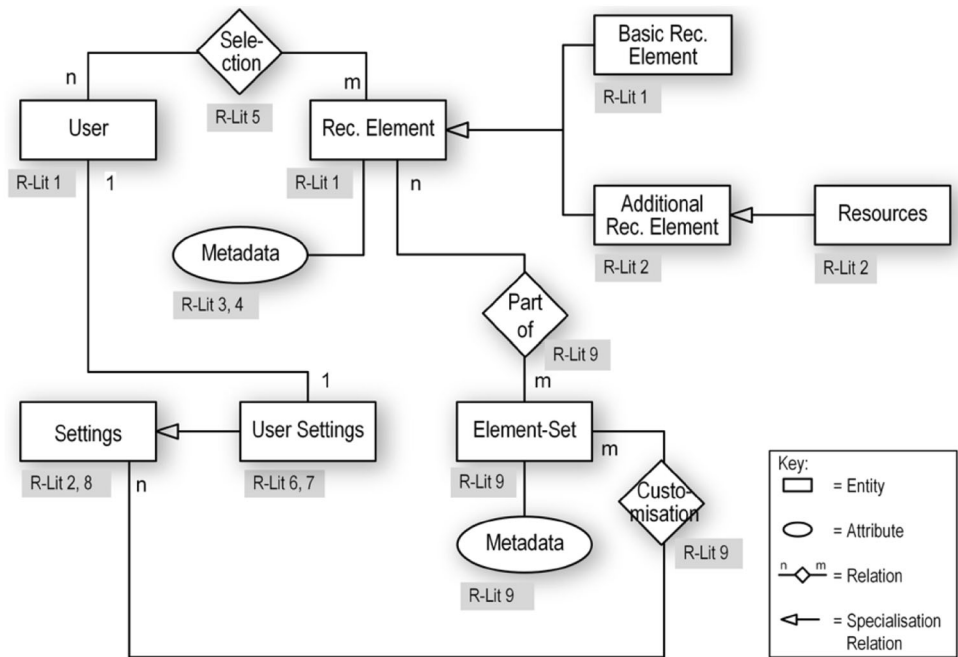
are constructed by using the well-established notation of Entity Relationship Diagrams that supports the specification of data structures on a conceptual level.

Content-Related Requirements

- R-Lit-1. *Recommendation of basic process model constructs.* The system should be able to recommend constructs such as elements, their structure and labels.
- R-Lit-2. *Recommendation of additional process model constructs.* The system should provide recommendations for other constructs such as resources.
- R-Lit-3. *Provide descriptive meta-information about the recommendations.* The system should provide relevant meta-information about the suggested elements.
- R-Lit-4. *Provide provenance information about the recommendation.* The system should provide information to assess the quality of the recommendation.

We implement R-Lit-1 by introducing the entity *Basic Recommendation Element*, R-Lit-2 by introducing the entity *Additional Recommendation Elements* (Fig. 2). We generalize both by introducing a more abstract *Recommendation Element*-entity. In order to capture descriptive meta-information about elements that can also comprise provenance information (i.e., source of origin) as demanded by R-Lit-3 and R-Lit-4 respectively, we introduce the attribute *Metadata* and associate it to the entity *Recommendation Element*.

Fig. 2 Literature-based partial model



Functional Requirements

- R-Lit-5. *Use personalization mechanisms.* The system should provide personalized recommendations tailored to the needs of the specific user in his or her specific modeling situation.
- R-Lit-6. *Adjustable filtering options for recommendations.* The user should be able to adjust the filtering criteria for recommendations.
- R-Lit-7. *Adjustable amount of recommendations.* The user should be able to adjust the amount of recommendations.
- R-Lit-8. *Multiple recommendation strategies.* Recommendations should be determined by using different calculation strategies in order to fit the user's requirements.
- R-Lit-9. *Support knowledge base evolution.* The system should provide capabilities such as versioning, change management, importing new content or learning.

R-Lit-5 can be embedded in the data model by introducing the relation *Selection* between the entities *Recommendation Element* and *User*. In this way, decisions to include a recommended element in the model made by a user are recorded as “selections” and can be leveraged for computing future recommendations.

R-Lit-6 and R-Lit-7 are functional requirements allowing the user to adapt the system to his or her individual preferences. Hence, an entity *User Settings* is introduced as a specialized form of a more general *Settings*-entity. Since selecting the best recommendation calculation technique as

required by R-Lit-8 may be a matter of experience, it is preferable to assign this task to an expert user engaged in setting up the system. So, R-Lit-8 is attributed to this more general *Settings*-entity. We reflect R-Lit-9 by introducing a separate entity *Element-Set* that represents an arbitrary number of collections of elements (e.g., process activities to be suggested for several business domains). In addition, to provide for versioning and change management capabilities, the attribute *Metadata* is associated to *Element-Set*.

Non-Functional Requirements

- R-Lit-10. *Ensure recommendations with a high semantic quality.* The system should provide recommendations that are adequate and lead to a high semantic model quality.
- R-Lit-11. *Flexible and easy application of recommendations.* The system should make it easy to work with recommendations and should guide the user in the selection of suggestions.
- R-Lit-12. *Compatibility to existing tools and languages.* The system should work with existing modeling languages and in conjunction with existing tools.

The semantic quality of recommendations is mainly dependent on the concrete algorithm used to calculate recommendations as well as on the concrete instance data, implying that R-Lit-10 is not directly relevant for the data model. Likewise, R-Lit-11 cannot easily be reflected in the data model on account of its non-functional nature (e.g.,

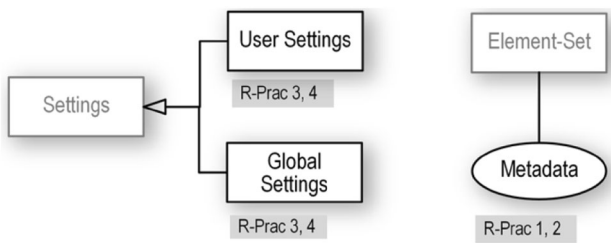


Fig. 3 Practitioner survey-based partial model

due to terms such as “flexible” and “easy”). Finally, R-Lit-12 demanding compatibility to existing tools is not eligible for representation in the data model. Figure 2 shows the derived partial model from the requirements along with the requirements that are depicted as graphical annotations.

3.2 Implementation of Requirements from a Survey Involving Practitioners (R-Prac)

The following requirements have been derived based on a study that involved 47 participants with practical modeling knowledge. The respondents have used the ARIS platform (40.4%), MS Visio (31.9%), Signavio (14.9%), Adonis (8.5%) and various office products (multiple selection was possible). The most commonly used languages were EPC (78.7%), BPMN (65.9%) and UML (44.7%). Regarding the background of the participants, almost half of them either were practitioners (40.4%) or had another background (10.6%) while the others had a research background. For more details, see (Fellmann et al. 2015b). Overall, the questionnaire has been answered by 47 people, 48.9% being researchers (23 persons), 40.4% practitioners (19 persons) and 10.6% others (5 persons, e.g., people in education).

- R-Prac-1. *Various sources for recommendations.* The recommendation system should be able to generate recommendations from various sources.
- R-Prac-2. *Provenance information.* The recommendation system should provide background information regarding the source and quality of a recommendation.
- R-Prac-3. *Display of recommendations on request.* Recommendations should be provided when the user requests the system to do so.
- R-Prac-4. *Multiple ways of displaying recommendations.* The recommendation system should provide multiple ways of displaying the recommendations varying in their degree of non-obtrusiveness.

We implement R-Prac-1 and R-Prac-2 by introducing an additional attribute *Metadata* and associating it with the entity *Element-Set*. In this way, the source and the origin of a recommendation can be recorded and exploited by the algorithm computing the recommendations. Adjustments to the provision of suggestions that a user or system administrator might want to set according to R-Prac-3 and R-Prac-4 can be stored in the *User Settings* and *Global Settings* respectively. Figure 3 shows the derived partial model from the requirements.

3.3 Implementation of the Requirements from the Case Study (R-Case)

Requirements have been collected in a case study involving 100 participants. In the case study, 52 participants were involved in 2013 and 48 participants in 2014. The participants were undergraduate students in the 6th semester with an equal share studying Economics and Information Systems. The participants had to create BPMN models in 11 groups with 5–6 participants in each group. During the modeling process, the participants had to use a standardized pre-defined set of process activities. Although the activities were not automatically suggested but had to be retrieved manually by browsing an extensive taxonomy of functions, this allows us to shed light on how users perceive modeling with pre-defined process activities. More details are described in (Fellmann et al. 2015b).

- R-Case-1. *Understandable recommendations.* Since one main positive aspect of using standardized activities is that their interpretation is less ambiguous, the PMRSs should use such standardized activities.
- R-Case-2. *Recommendation of “uncommon”, innovative contents.* For example, the system may suggest activities that are executed typically in a different industry and thus inspire the process design.
- R-Case-3. *Extension capability of the pre-defined contents.* To provide a remedy for missing activities, the recommendation system should include a feature to extend the internal knowledge base.
- R-Case-4. *Benchmarking feature.* The system should facilitate benchmarking, e.g., by suggesting Key Performance Indicators (KPI) or by enabling a comparison of KPI values.

- R-Case-5. *Advanced model processing features.* The system should offer advanced features for the translation of models in multiple languages (e.g., process taxonomies such as PCF exist in different languages), to compare models or to show which area of enterprise activities they cover based on their semantics.
- R-Case-6. *On/off switch and decent presentation of recommendations.* The system should be switched off easily and the recommendations should be presented decently.

We reflect R-Case-1 by the attribute *Metadata* associated to the entity *Recommendation Element* since additional information stored as metadata might help to provide understandable recommendations. Their “uncommonness” or innovativeness as requested by R-Case-2, however, is an attribute of the data being stored in the *Element-Set* and as such cannot be reflected in terms of structures in the data model. Further, import features for new content as demanded from R-Case-3 as well as advanced model processing features required by R-Case-4 have to be implemented as part of the functionality of a PMRs and thus are not reflected in the data model. In order to provide a benchmarking feature as implied by R-Case-4, Key Performance Indicators are required since they form the basis of any comparisons. These factors can be suggested similar to other *Additional Recommendation Elements* and thus are introduced as a specialization of the latter. An on/off-switch required by R-Case-6 can be realized as part of the *User Settings* data. Figure 4 shows the derived partial model from the requirements.

3.4 Implementation of the Requirements from Assessing a Prototype (R-Prot)

This section describes the implementation of the following requirements collected by assessing a prototype with 66 business users. The assessment was achieved by presenting a prototype to users at a major industry fair (CeBIT) during 4 days. The prototype system comprised a simple web application capable of offering modeling suggestions for modeling sequences of activities. After the demonstration, an interview and survey was conducted that both focused on requirements and user needs. More details are described in (Fellmann et al. 2015b).

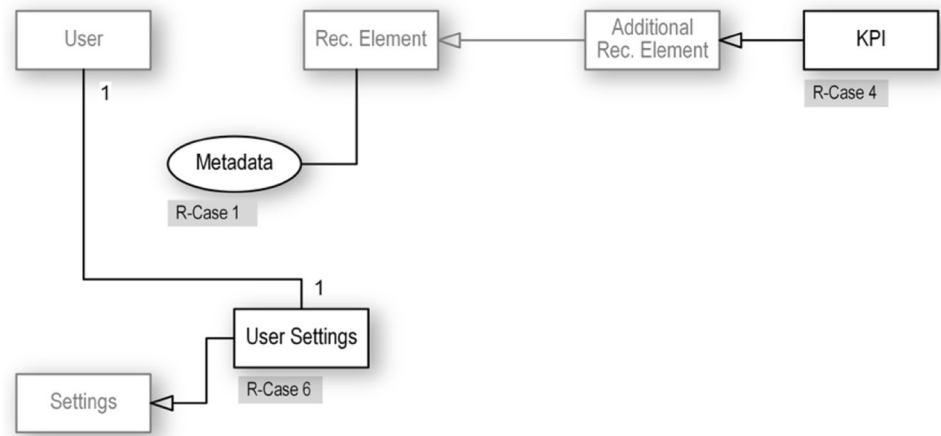
- R-Prot-1. *Recommendation of organizational units.* The system should recommend additional elements such as organizational units executing the activities.
- R-Prot-2. *Recommendation of resources.* The system should recommend resources such as documents, tools or information systems.

- R-Prot-3. *Customized specific taxonomies.* To make sure a plethora of potential use cases is covered, the predefined contents in the system should be customizable.
- R-Prot-4. *Mobile version of the recommender.* Due to the fact that an increasing amount of work is not done from the office, a mobile version should be offered.
- R-Prot-5. *Interface to other systems.* Data inside the PMRs used for recommendations such as taxonomies of pre-defined activities or organizational units should be updated frequently via interfaces to systems containing that data.
- R-Prot-6. *Support multiple platforms.* As there are different platforms and architectures used in companies the support of the most important of them is crucial to ensure the system gains acceptance.
- R-Prot-7. *“Intelligent recommendations”.* This requirement is rather an overall characteristic of the whole system and implies that recommendations should be made at the right time and in the right manner with adequate content.
- R-Prot-8. *Show recommendation context.* The user of the system should be informed about the semantic context of a recommendation that is offered.

The requirement of suggesting additional elements as stated in R-Prot-1 and R-Prot-2 can be implemented by introducing the requested elements as specialized *Resource* entities *Documents*, *Tools* and *Information Systems*. The requirement of customizing the element collection used to calculate recommendations stated in R-Prot-3 can be reflected by the *Part-of*-relation between *Recommendation Elements* and *Element-Set*. In this way, the same recommendation elements can be part of different element sets (e.g., an element set for each industry the recommendation tool is used in). The usage of the PMRs in a mobile version (R-Prot-4) and its interfaces to other systems (R-Prot-5) as well as support for multiple platforms (R-Prot-6) are outside the scope of the data model. Also, “intelligent” recommendations (R-Prot-7) are an obligation of the algorithm that operates on top of the data. Further, showing the recommendation context as requested by R-Prot-8 can be supported using the information stored in the attribute *Metadata* associated to the *Recommendation Element*-entity. Figure 5 shows the derived partial model from the requirements.

3.5 Preliminary Integrated Data Model

The partial models developed in the previous sections are integrated into a single model depicted by Fig. 6.

Fig. 4 Case-study-based partial model

4 Critical Review and Refinement of the Preliminary Integrated Data Model

In this section, we first contrast our preliminary integrated data model with a real-world data model resulting from the implementation of a PMRS. Then, we check the general ability of our integrated data model to satisfy data demands known from established recommender system paradigms. Both sections lead to refinements of the data model presented in its final form in the last sub-section.

4.1 Contrasting the Requirements-Based Synthesized Model with a Real-World Data Model

The real-world data model that we use in the following to contrast our developed model originates from a real prototype. The prototype was built in the frame of the SEMPHIS-project using web technologies on the frontend in conjunction with a server-based backend (PHP, MySQL). With this tool, a modeler is able to create simple linear process models using the recommending and autocomplete-feature of the system. The auto-completion feature allows completing the labels of model elements based on keywords typed in an input field. The recommendation part of the system is capable of suggesting the next element (cf. Fig. 7 – generated successor suggestions for the previously inserted model element “Define strategic standards, guidelines, and principles”).

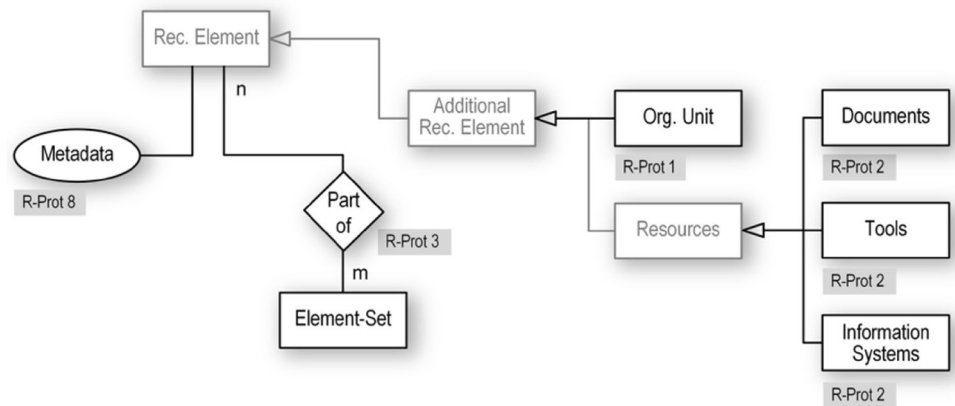
The system was populated with process information from the Process Classification Framework (PCF) in version 6. PCF is a reference model that consists of approx. 1000 business activities organized in 12 areas and 4 levels of hierarchy. It is available in a generic form as well as tailored to specific industries such as aerospace, automotive, banking, broadcast, consumer products, education, electric utilities, pharmacy, petroleum, and telecommunications industry. However, other sources such as the

Enterprise Ontology or other ontologies discussed in the context of semantic business process modeling might also be used (Thomas and Fellmann 2009). Moreover, lists of business activities can also be derived from enterprise systems such as Enterprise Resource Planning (ERP), e.g., SAP ERP.

The ranking of the suggested model elements is computed using four parameters: *semantic match*, *industry*, *department*, and *popularity*. *Semantic match* is a measure computed using lexical string processing features. It reflects how well the label of a model element matches to an element in the PCF taxonomy which serves to identify a possible next element (e.g., if the label best matches to an PCF element 6.2.5 then element 6.2.6 in the taxonomy would be a suggestion candidate; if 6.2.6 would not exist, then the first element of the next subcategory 6.3 would be used as candidate). *Industry* and *department* serve to reward suggestion candidates in the ranking that match the user settings in regard to industry (e.g., automotive) and department (e.g., HR). *Popularity* is calculated as the (global, i.e., not user-dependent) ratio of displayed suggestions to accepted modeling suggestions, e.g., if an element is selected from the user every time when it is suggested, this value would be 1. The weight of all parameters used for calculating the ranking can be adjusted by the user.

The tool was implemented initially to serve as a testbed and evolved over time. Hence, the systematically derived data model was not present in the beginning of the tool development. This is why the data model of the real-world tool provides a good opportunity to review and refine the requirements-based synthesized data model leveraging the experiences from a real-world implementation. Figure 8 shows the data model which we reverse-engineered for the sake of our data model development. Shaded elements indicate the recommendation-specific parts, whereas non-

Fig. 5 Demonstration-based partial model



shaded parts represent data needed for more general modeling functionality.

The following description (in *italics*) of the data model was given by the person who was involved in its development and charged with managing the data model. References to elements of the implemented PMRS inside the description are included with single quotation marks “”. We interrupt the description to comment on missing parts in our data model and arrange these comments by enumerated paragraphs. For brevity, we do *not* comment on equivalences between the two models.

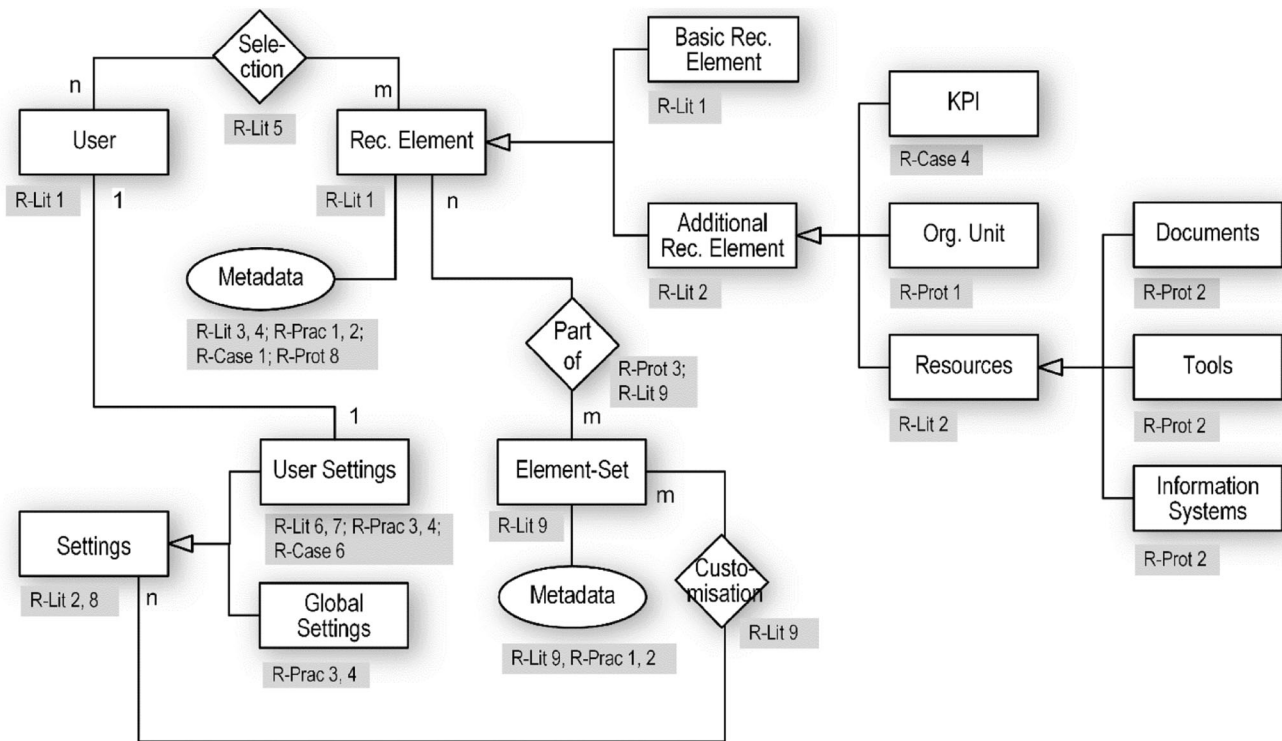
“The central aspect of the conceptual data model is the ‘element’ entity. It represents the central aspect for saving information about potential elements that could be used in a recommender system. Such information can comprise, e.g., the name, industry, department or other meta-information including a counter of how often an element has been displayed and accepted from users. As the recommendation is mainly based on names, the attribute ‘name’ is important. Furthermore, as in mostly all entities, the use of an ID as primary key is compulsory. As one of the requirements was to allow any kind of hierarchies, the entity ‘hierarchytype’ became part of the conceptual data model. First, information about what kind of hierarchy can be defined such as superior, inferior, equal, parent or child relations. In combination with the relation ‘elementhierarchy’, which combines two entities ‘element’ and an entity ‘hierarchytype’, the saving of generic hierarchies becomes possible. The design with the generic entity and the ternary relation is necessary to make sure that individual terminologies as well as different kinds of relations are possible. Potentially, the generic construct could be replaced with individual relations named after the kind of hierarchy (e.g., three relations named ‘follower’, ‘parent’ and

‘child’). However, in a conceptual data model, generalization is preferred.”

- (1) In our current data model, we cannot represent relations such as hierarchies or follower-successor relations. We therefore have to extend the model with an entity *Relation Type* and a ternary relation *Element Relation*.

“Another essential requirement of a generic conceptual data model is the need to save relevant metadata about an element such as the industry for which the element is specific, the context in the enterprise (e.g., production, HR), etc. To make sure that all potential values for arbitrary metadata are storable, this has been included in a generic way. Therefore, a new entity ‘attributetype’ is introduced. This entity represents meta-information about what types of information could be saved. Consequently, this entity will hold the name of the information (e.g., industry, context, etc.). The information itself will reside in the attached relation between the ‘attributetype’ and the ‘element’ which is called ‘elementattribute’. It includes the value and, untypical for a relation, an ID as primary key. This makes sure that for a defined ‘element’ and for one ‘attributetype’, more than one ‘value’ is storable (e.g., multiple industries for an activity).”

- (2) In our current data model, we cannot store metadata about *Recommendation Elements* and *Element-Sets* in a generic way. We therefore refine our data model and switch *Metadata* from an attribute to a separate entity type that is connected to both *Recommendation Elements* and *Element-Sets* via a relation. In this way, a generic mechanism for storing metadata is possible.



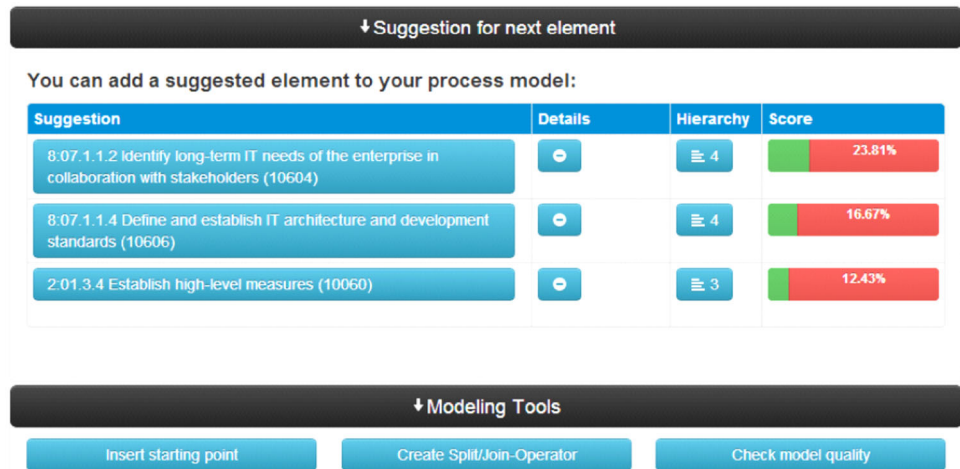
“For a modeling environment, the central entity is the model with its very own data such as ‘title’ and ID and potentially many more. It is connected through the relation ‘modellink’ to the entity ‘modelement’ which saves the elements within the model with their name and with an ID. However, from the central entity ‘element’ a relation was built which bridges the recommending to the modeling system. To make sure that certain ‘modelements’ that got inserted into a ‘model’ do not lose their link to an element proposed by the recommender this link is saved within the relation ‘elementlink’. So, after inserting a recommended element into a model the conceptual data model will be able to record this.”

- suggestable elements. This provides the option to recommend objects.*

Another part of the conceptual data model is a relation called ‘modelattribute’ between a ‘model’ and an ‘attributetype’. The objective of the relation is to save information for a whole ‘model’ in the same way an ‘element’ can have attributes of (e.g., industry, context, etc.). This guarantees that potential recommendation algorithms can take account of model information and adapt their recommendations.”

- "Equally as to the 'modelattribute', the conceptual data model was extended to the relation 'userattribute' which connects the user from the modeling system with the 'attributetype'. For the same reason of saving special information about a model, the system is enabled to save information about the user (e.g., the user is from a special industry, context, etc.). This can also be integrated in the*

Fig. 7 User interface of the PMRs implementation



recommending system to make sure that the recommendations are adapted accordingly.”

4.2 Checking the Compatibility with Established Recommender System Paradigms

An important question regarding the versatility and broad applicability of the integrated data model is whether it has the capability to satisfy data demands originating from established recommender system paradigms. Such paradigms and the data they involve are described, e.g., from Jannach and Ricci (Jannach 2011; Ricci et al. 2011). According to an empirical investigation (Jannach et al. 2012), collaborative filtering, content-based filtering, knowledge-based filtering and hybridisation paradigms are most commonly used. Within the discipline of Information Systems, collaborative and knowledge-based filtering are the dominating paradigms. In the following, we quote the description of each paradigm as given by (Ricci et al. 2011) and comment on how the data requirements and needs implied by each paradigm can be fulfilled using entities and relations of our integrated data model. Referenced entities and relations are indicated using a sans-serif font.

Content-based filtering

“The system learns to recommend items that are similar to the ones that the user liked in the past. The similarity of items is calculated based on the features associated with the compared items. For example, if a user has positively rated a movie that belongs to the comedy genre, the system can learn to recommend other movies from this genre.”

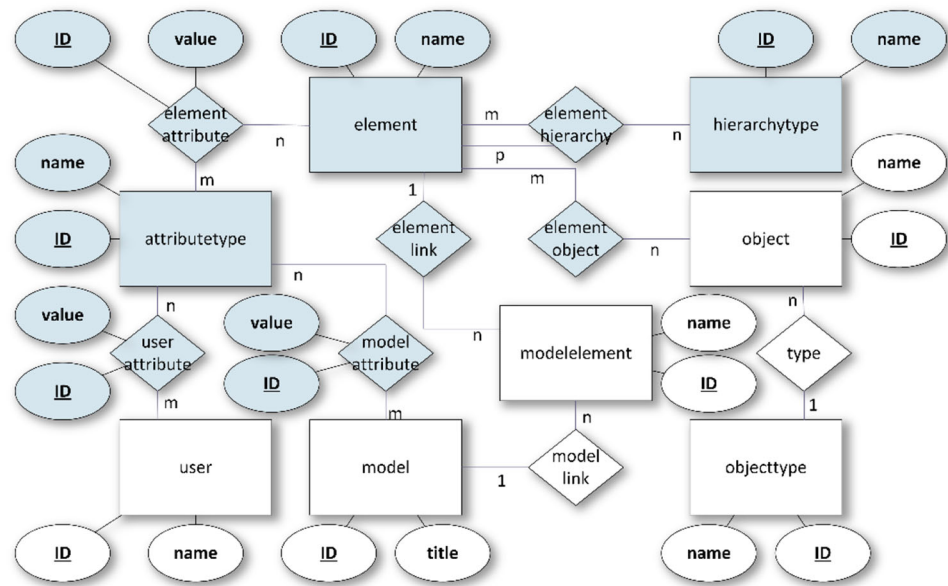
Using our data model, content-based filtering can be implemented in such a way that the system learns to suggest **Recommendation Elements** that are in some

criteria similar to the ones that the **User** added in the past to a **Model**. For example, if it is recorded in the **Selections** entity that a user has recently added a **Recommendation Element** “Check customer solvency” that belongs to a collection of distribution-related tasks and more specifically to tasks addressing first-time customers, then the system can recommend other **Recommendation Elements** also relevant for processing first-time customer orders.

Collaborative filtering

“The simplest and original implementation of this approach [...] recommends to the active user the items that other users with similar tastes liked in the past. The similarity in taste of two users is calculated based on the similarity in the rating history of the users.”

Using our data model, collaborative filtering can be implemented in such a way that the system suggests to the active User the **Recommendation Elements** that other users added to a model in the past. The similarity of two users is calculated based on the data stored in the **Selection** entity, i.e., based on a similar modeling history. In case of few users, the similarity function could be implemented in such a way that instead of users, *user roles* (e.g., manager, process user, IT staff) are matched. User roles could be implemented as an attribute **User Type** of the **User** entity. In this way, if a manager, having, e.g., a **User Type**-value of “1”, creates a model, he or she will be served with suggestions of **Recommendation Elements** that other users with **User Type** of “1” have selected in the past. Another possible variation of collaborative filtering would be to suggest to the active User the **Recommendation Elements** that were added to similar **Models**. The similarity of two models is calculated based on the data stored in the **Model Element** entity.

Fig. 8 Data model of the real-world PMRs

Knowledge-based

“Knowledge-based systems recommend items based on specific domain knowledge about how certain item features meet users needs and preferences and, ultimately, how the item is useful for the user. [...] In these systems, a similarity function estimates how much the user needs (problem description) match the recommendations (solutions of the problem).”

Using our data model, knowledge-based recommendations can be implemented based on domain knowledge about how certain **Recommendation Elements** features meet users needs and preferences. Data about these features can be stored in the **Metadata** entity. Examples for this would be provenance information as well as the department and type of industry for which an element is relevant. Data about preferences can be stored either in the **User Settings** entity (e.g., prefer elements relevant for the role “manager”), in the **Global Settings** entity (e.g., prefer elements that conform to a standard) or in the **Model Settings** entity (e.g., prefer elements relevant for the sales department in the automotive industry). Furthermore, advanced knowledge-based recommendations can be made by exploiting **Element Relation** and **Relation Type**. For example, if it is recorded in the **Selections** entity that a user in a previous modeling step has added a **Recommendation Element** “Check order”, then the system can suggest a next element “Notify customer” if an **Element Relation** of **Relation Type** “successor” is stored in the data that connects the two Recommendation Elements. In a similar way, by exploiting these data structures, resources can be suggested as well.

Hybrid recommender systems

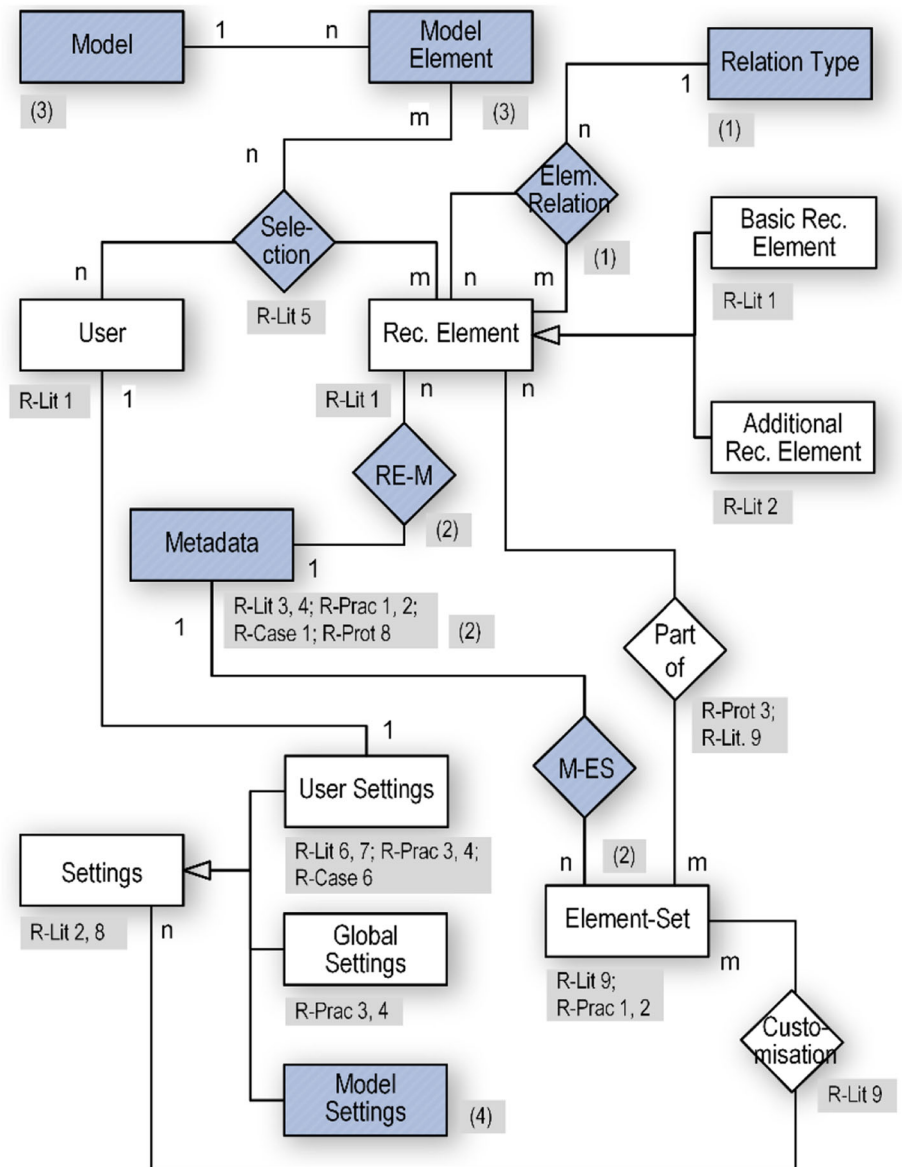
“These RSs are based on the combination of the above mentioned techniques. A hybrid system combining techniques A and B tries to use the advantages of A to fix the disadvantages of B. For instance, collaborative filtering methods suffer from new-item problems, i.e., they cannot recommend items that have no ratings. This does not limit content-based approaches since the prediction for new items is based on their description (features) being typically easily available.”

Using our data model, there are no limits in mixing the paradigms. For example, a content-based approach that filters model elements according to specified preferences in terms of the role, department, industry and provenance of elements can be combined with collaborative filtering adding a “social” component to the recommendations.

All in all, our integrated data model is compatible with the presented established recommender system paradigms and is eligible to meet the data demands implied by these paradigms.

4.3 Final Refined Data Model

In the following, we present the final refined data model (cf. Fig. 9) that implements the changes identified above. We further provide a list of all entities and relations in the final refined data model (cf. Table 1). In this list, we provide the name of the element, its type (“E” for entity, “R” for relation), and source model from which it originates. In the last column, we briefly describe the modifications and simplifications that have been made for the final model.

Fig. 9 Final refined data model

Finally, we briefly characterize our consolidated model from a high-level perspective. Regarding its content, entities linked to three aspects are included: (1) Managing the *content of recommendations* such as the recommendation elements and their interrelation as well as metadata such as provenance data, (2) specifying *settings for recommendations* such as user, global or model-specific settings and finally (3) *storing context information of selected recommendations* which is the recommended element, the current user and for which individual model element a recommendation was computed. Regarding the growth of instance data that applications implementing the data model are expected to store while being used, it can be assumed that the last aspect seems to dominate the others. The volume of information regarding the selection of recommendations is expected to grow faster when the system

is in active use. In contrast, the volume of data associated to recommendation elements and user settings is more constant (at least, unless the system does not, e.g., change the set of recommendation items using learning techniques).

5 Exemplary Use Scenario

Today, the increasing digitalization of businesses leads to very fast developments of new business models and innovations. Enterprises need to be enabled to respond and adapt quickly to such changes and challenges. According to Forrester Research (Richardson and Miers 2013), there are ten dimensions that shape the agility of a digital business. Surprisingly, five out of them are process-oriented

Table 1 Origin and modification of elements and relations in the final model

Element	Type	Source Model(s)	Revisions
Model	E	Real-world tool	Inserted in the final model due to issue (3) raised in Sect. 4.1
Model Element	E	Real-world tool	Inserted in the final model due to issue (3) raised in Sect. 4.1
Selection	R	Literature-based	In the final model, this relation has been connected to the element “Model Element” due to issue (3) raised in Sect. 4.1
Relation Type	E	Real-world tool	Inserted in the final model due to issue (1) raised in Sect. 4.1
Element Relation	R	Real-world tool	Inserted in the final model due to issue (1) raised in Sect. 4.1
User	E	Literature-based	None
Rec. Element	E	Literature-based	None
Basic Recommendation Element	E	Literature-based	None
Additional Recommendation Element	E	Literature-based	For the sake of simplicity, the following specializations of this entity have been omitted: KPI (from case-study-based model), resources such as documents, tools and information systems (all from literature-based model) as well as organizational units (from demonstration-based model)
Relation RE-M	R	Real-world tool	Inserted in the final model due to issue (2) raised in Sect. 4.1
Metadata	E	Literature-based, practitioner survey-based-based, case-study-based, demonstration-based, real-world tool	Two formerly separate metadata attributes related to Rec. Element and Element-Set have been merged into a single Metadata entity according to issue (2) raised in Sect. 4.1
Relation M-ES	R	Real-world tool	Inserted in the final model due to issue (2) raised in Sect. 4.1
Element-Set	E	Literature-based, demonstration-based	None
Part of	R	Literature-based	None
Settings	E	Literature-based	None
User Settings	E	Literature-based, practitioner survey-based-based, case-study-based	None
Global Settings	E	Practitioner survey-based-based	None
Model Settings	E	Real-world tool	Inserted in the final model due to issue (4) raised in Sect. 4.1
Customization	R	Literature-based	None

dimensions. Forrester also states that for being agile, awareness and execution are most relevant and important factors. The multitude of conceivable application areas of mobile and wearable devices – for instance in flexible production processes as presented in the example – leads to frequent process reconfigurations, which in turn need to be handled in an agile manner. In this context, process modeling recommender systems are representing an innovative technology to achieve business agility and they therefore deserve our attention. Towards this end, we devise a prototypical application scenario that builds upon the identified dimensions and that instantiates our data model in a mobile modeling environment.

5.1 Example Case: The Smart Glasses Scenario

In this case, a Smart Glasses-based service support system (Metzger et al. 2017) was chosen to demonstrate the applicability of the model. Smart Glasses are wearable computers that typically provide an information overlay to

what the wearer sees. Especially in the domain of technical services, mobile and wearable information systems have been a frequent subject of discussion to support the execution of technical services such as maintenance or repair (Starnier et al. 1997; Henderson and Feiner 2011; Fellmann et al. 2015a). For service support, Smart Glasses are able to provide step-by-step guidance throughout the service process and thus fulfil capabilities of a mobile process aware information system. Furthermore, they provide additional forms of interaction such as voice commands (Metzger et al. 2017). In recent work, the run-time modeling of service processes in parallel to their execution has been discussed as an innovative feature being enabled by this technology. Exemplarily, Niemöller et al. (2017) found Smart Glasses-based service process documentation to be of high value for businesses in the logistic sector. Similarly, Metzger et al. (2017) propose a system architecture for a Smart Glasses modeling environment to tackle emerging challenges in the domain of technical customer service, while Jannaber et al. (2017) conceptualizes the

development of a process modeling language specifically tailored for Smart Glasses-based process modeling. The aforementioned contributions correspond to suggestions made in (Weber et al. 2008) and (Lerchner and Stary 2016), who elaborate on run-time modeling as an approach for process discovery and increased modeling flexibility which addresses difficulties that come along with frequent process changes and outdated models. The integration of a process recommender system into the previously characterized Smart Glasses-based environments for run-time process modeling in the context of technical services is particularly beneficial due to multiple reasons: (a) Technical service processes are inherently complex (Blinn and Nüttgens 2010; Matijacic et al. 2013). Consequently, the manual modeling of service processes requires an increasing amount of effort and is prone to modeling errors. Process recommender systems are able to partly automatize and simplify the model construction process via customized suggestions. Furthermore, technical services are characterized by an increasing need for information support. In addition, interactions with supporting devices need to be hands-free to not interfere with service execution (Niemöller et al. 2016). To this end, Smart Glasses have emerged as a particularly promising solution to support technical service execution (Niemöller et al. 2016) (b) End-users, such as service technicians, are domain experts but lack modeling skills and in this regard can be considered as “novice modeler” (Wilmont et al. 2010). Hence, applying common process modeling languages and creating high quality models is a difficult and time-consuming task, especially when carried out in parallel to the actual service execution. Using process recommenders, end-users are able to create valid process models without a deep understanding of the underlying modeling language. Additionally, cognitive effort is reduced, for example due to label recommendations. (c) Modeling languages are getting increasingly difficult (zur Muehlen et al. 2007). The increasing complexity of process modeling languages leaves process modeling a topic for domain experts. The complexity becomes even more striking considering the fact that the modeling task has to be done in parallel to service execution. A process modeling tool augmented by a recommender system feature faces the outlined language complexity issues by providing sophisticated modeling recommendations that maintain the expressiveness of a modeling language even when used by a novice modeler. To do so, the system should be designed in such a way that syntactically correct and semantically meaningful modeling suggestions are offered. For example, no deadlock should be introduced in a process model once the user accepts a modeling suggestion. However, its implementation is application-specific.

On the basis of the reasons given above, we argue that the outlined application scenario is especially suited to demonstrate the applicability and value of a process recommender system. We therefore select this as an example application case to illustrate an instantiation of our data model. In detail, our scenario features a maintenance process that is being modelled in a Smart Glasses-based modeling environment. The modeling task is supported by a process recommender system operating on the final data model presented in Sect. 4.3. In the following, an instantiated excerpt of the data model is shown with exemplary data. The alignment of data model and process recommender results highlights the way data is stored and processed according to the proposed data model.

5.2 Exemplary Instantiation of the Recommender Data Model

The application scenario that we unfold in the following features a technical service in the field of engine maintenance. In parallel to its execution, the maintenance process is being modelled by a service technician for documentation purposes using a Smart Glasses modeling environment and the Event-driven Process Chain (EPC) modeling language. The process recommender system operates on the proposed data model to facilitate the modeling tasks with recommendations of adequate elements such as functions or events. For a prototypical instantiation, we applied a knowledge-based recommendation approach as stated in Sect. 4.2 and depicted the corresponding data model entities as relational data tables. By doing this, potential data entries are visualized, but also emphasis can be put on necessary attributes that may be stored within each table. The recommendation of process elements and required data model entities within a Smart Glasses-based modeling environment is visualized in Fig. 10.

Essentially, the visualization is divided into three layers: The left side visualizes the Smart Glasses screen that displays information and commands available to the user. In the center, the EPC model to be created is depicted. The relational data tables of the *User*, *User Settings*, *Element Set*, *Basic Rec. Element*, *Additional Rec. Element* as well as both of their *Metadata* and *Element Relation* entities are aligned to demonstrate their usage within a knowledge-based process recommender system. The interrelation of the data tables depicted in Fig. 10 will be introduced from the point of view of the fictional service technician “Albert”.

In the presented application scenario, Albert has a unique user account stored in the *User* table with assigned roles that determine the user’s rights to navigate within the system. Here, Albert is able to customize the process recommender system towards his specific needs, which are

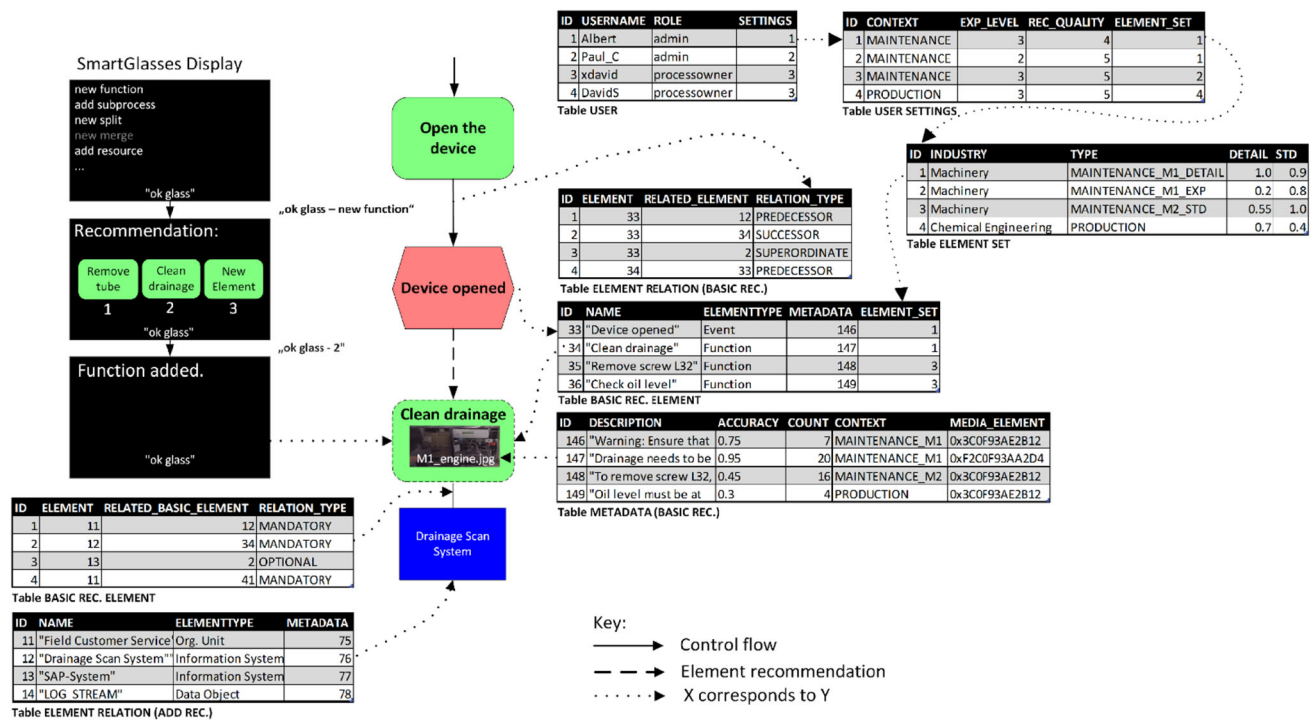


Fig. 10 Smart glasses recommendation-enabled modeling environment

being stored within the *User Settings* table. Besides the experience level or the desired recommendation quality (e.g., detailed process steps vs. high-level overview), each user setting entry is associated to a specific set of process elements to be recommended included in the *Element Set* table. In the given case, the element set with the ID 1 is associated to Albert's user settings, since this set fits his preferences regarding industry context (Machinery) and recommendation quality (Detail = 1.0). Henceforth, only elements that are included in the element set with the ID 1 will be recommended to Albert. Figure 10 shows that Albert has already modelled an EPC maintenance process of arbitrary length using a Smart Glasses-based modeling environment. The process recommender system is triggered, since Albert needs to add a new element into the model. Via voice command "new function", Albert interacts with the modeling system and initiates the insertion of a new EPC function after the event "Device opened". For process element recommendation, the most important entity is the *Basic Recommendation Element*. The corresponding relational data table holds historical data from the system's knowledge base of all process elements modelled previously and thus can be used for potential recommendation. As such, the event "Device opened" is already included in the database, as it has been used by a colleague of Albert in a related maintenance process model. For each element included in the *Basic Recommendation Element* data table, its relations with other process elements are

captured using the ternary relationship type *Element Relation*. Accordingly, predecessor/successor relationships as well as element hierarchies (in case of sub-processes) can be derived deductively from the table *Element Relation (Basic Rec.)*. Subsequently, the recommender system in the presented scenario is able to identify all element relationships involving the event "Device opened" with specific focus on each stored succeeding relationship. The tables featured in Fig. 10 demonstrate that a potential successor for the event "Device opened" is a function labelled "Clean drainage" which is also included in the same element set as "Device opened". Furthermore, the function yields a high recommendation accuracy, since it has already been recommended 20 times of which it has been approved and selected 19 times ($19/20 = 0.95$) by other service technicians in the past.

Accordingly, the function "Clean drainage" is provided to Albert as a recommendation for the next process element. Thus, the given case represents the selection of recommendations, i.e., in Fig. 10 the functions "Remove tube" (not featured within Fig. 10) and "Clean drainage", on the basis data stored in the *Metadata* (recommendation accuracy) and *Element Relation* (predecessor/successor relationship of elements included in the same element set). However, advanced matching algorithms such as string and pattern matching can be applied as well to provide high quality recommendations. Exemplarily, string matching techniques may be used to assist label recommendations

for new process elements, whereas pattern matching algorithms are able to calculate recommendations on the basis of relationship pattern included in the database. In the application scenario, Albert selects the recommendation that best fits his current service task. However, in any case, the system provides the opportunity to insert a new function in case no recommendations can be suggested or the recommendation quality is low. Figure 10 demonstrates that the recommended elements are displayed in the Smart Glasses interface. Albert choses the function “Clean drainage” via the command “ok glass, -2”. Subsequently, the function is inserted into the process model in combination with an image file that is stored as an associated media element in the *Metadata* table.

In addition to EPC functions or events, the process recommender data model also provides for additional recommendation elements to be suggested to the user. In the EPC case, additional recommendation elements may be resources that can be attached to functions. Similarly to basic recommendation elements, the respective data table contains previously modelled resources as well as their relationship to functions and metadata. While in the given case metadata for resource elements have been left out intentionally, the table *Element Relations* enables a recommending system to identify a “Mandatory” relationship between the chosen function “Clean drainage” and an information system labelled “Drainage Scan System”. Due to this relation type, the resource is automatically added to the recommended function “Clean drainage”. However, in case of an optional relationship, the user can be given a choice whether to include the recommendation. As an outlook on further recommendation capabilities, the data model enables process recommender systems to suggest additional resources via string matching of functions. Exemplarily, if a function labelled “Check drainage” with an attached resource “Drainage Checklist” (not featured within Fig. 10) is already included in the databases, the recommender system can detect label similarities between the existing function “Check drainage” and a potential novel function named “Clean drainage” and subsequently suggests the resource “Drainage checklist” to also be attached to the newly modelled function “Clean drainage”.

The process recommender as presented in Fig. 10 represents a system extension that is currently being implemented to complement a run-time process modeling environment specifically tailored for Smart Glasses. Figure 11 visualizes the sequence of interfaces that are visible to the user when adding a new process function using the software prototype’s status quo without automatic recommendation functionality.

The recommendation extension is built upon the proposed data model and follows a knowledge-based

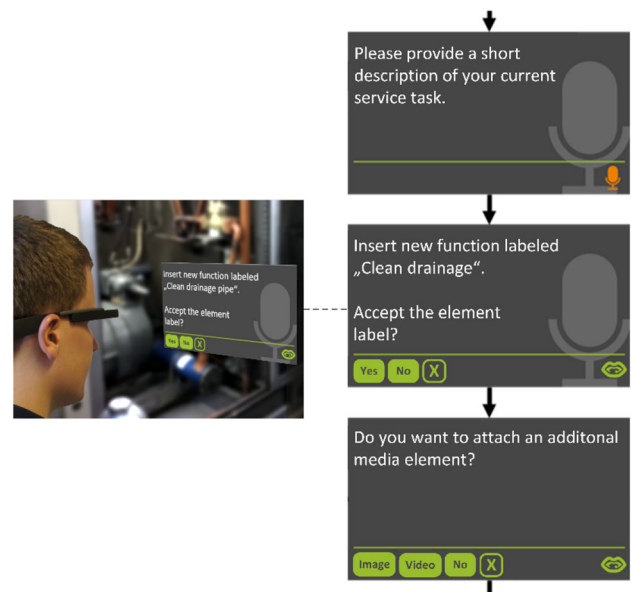


Fig. 11 Implementation of a run-time process modeling system with Smart Glasses

recommendation approach which makes use of existing process models already stored in the database. Hence, due to the introduced extension, the prototype is able to draw from the knowledge base in terms of element recommendations, while at the same time actively contributing to the knowledge base by storing modelled processes in the database. The primary objective of the recommender system is to facilitate the manual modeling of process elements of the prototype by reducing the complexity of run-time process modeling with automatic element suggestions. Referring to Fig. 10, the recommender extension simplifies the depicted workflow by offering the opportunity to select suggested elements that are deemed suitable by an appropriate recommending algorithm. The suggested elements already include a given label and come along with additional recommendations for, e.g., resources, which greatly reduce the manual effort necessary in the current state of the presented prototype. However, a manual insertion of novel process elements is still possible.

6 Summary and Outlook

Although sophisticated modeling tools exist, guidance in process modeling in terms of auto-completion and recommendation features is – apart from some few research proposals – largely missing even in today’s tools. However, to build such tools is a challenging task since a multitude of data and parameters have to be organized. Unfortunately, existing research works neither make the data structures underlying recommendation-based systems explicit nor are

the data structures developed in a systematic way. In this contribution, we therefore filled this gap by providing a systematically developed requirements-based data model. To do so, we used previously elicited requirements from various sources in order to construct partial models. We then combined these partial models to form an integrated model. In the next step, we critically reviewed our data model in the light of a data model that was obtained by analyzing a real-world system. This analysis led to interesting insights that in turn resulted in extensions and revisions of the initial integrated model. These insights would not have been possible without the experiences gained from the implementation of the real-world system. We also checked the compatibility of our developed data model with the data requirements of established recommender system paradigms. We hope that our data model will facilitate the implementation of new PMRS or serve as a basis for further development activities around existing recommender systems. In addition, we hope that our approach will inspire more research and development in the field of modeling support. Moreover, we are convinced that it will increase business agility from a process-oriented perspective. The time required to create or update business process models with recommender functionalities could be largely reduced, especially if the tasks and resources involved in the workflows are stable and modeling mainly serves the purpose of reconfiguring existing tasks in an agile way (e.g., in Industry 4.0-settings or highly individualized production). In this way, businesses can respond to changes faster and in a more flexible way, so that the approach should also have a positive effect on the speed to market or delivery.

Future research opportunities include exploring additional real-world data models of other types of recommendation or assistance systems in order to additionally refine our data model. Moreover, we plan to further develop and improve the algorithms working on top of the data model that have been demonstrated in the scenario of the Smart Glasses-based modeling environment. Similarly, the presented prototype needs to be applied in a real-world modeling scenario and assessed regarding the usefulness and feasibility of the run-time modeling concept. Finally, measuring the effects of recommendation-based modeling in terms of model quality and from an economic perspective will also be subject of future research.

References

- Blinn N, Nüttgens M (2010) Empowering technical customer services with intelligent mobile applications: a contingency theory-based approach. In: Proceedings of the IFIP 8.2 Organizations and Society in Information Systems Pre-ICIS Workshop, St. Louis, Working Papers on Information Systems 10 (107)
- Born M, Brelage C, Markovic I et al (2009) Auto-completion for executable business process models. Business process management workshops. Springer, Heidelberg, pp 510–515
- Clever N, Holler J, Shitkova M, Becker J (2013) Towards Auto-Suggested Process Modeling Prototypical Development of an Auto-Suggest Component for Process Modeling Tools. In: Proceedings of the 5th International Workshop on Enterprise Modeling and Information Systems Architectures, St. Gallen, pp 133–145
- Fellmann M, Heitmann C, Metzger D, Nobbe L, Thomas O (2015a) TKD 4.0 – Klassifikation, Einordnung und Bewertung der Einsatzpotenziale von Augmented-Reality-Anwendungen für den Technischen Kundendienst. In: Thomas O (ed): Living Lab Business Process Management Research Report No. 10, Osnabrück
- Fellmann M, Zarvic N, Metzger D, Koschmider A (2015b) Requirements catalog for business process modeling recommender systems. In: Thomas O, Teuteberg F (eds) Proceedings of the 12th International Conference on Wirtschaftsinformatik, Osnabrück, pp 393–407
- Fellmann M, Metzger D, Thomas O (2016) Data model development for process modeling recommender systems. In: 9th IFIP WG 8.1 Working Conference on the Practice of Enterprise Modeling, LNBIP 267, Skövde, pp 87–101
- Henderson S, Feiner S (2011) Exploring the benefits of augmented reality documentation for maintenance and repair. *IEEE Trans Vis Comput Graphics* 17(10):1355–1368. <https://doi.org/10.1109/TVCG.2010.245>
- Hornung T, Koschmider A, Lausen G (2008) Recommendation based process modeling support: method and user experience. In: Li Q, Spaccapietra S, Yu E, Olivé A (eds) Conceptual modeling - ER 2008. Lecture notes in computer science, vol 5231. Springer, Heidelberg
- Jannaber S, Zobel B, Riehle DM, et al (2017) Development of a domain-specific language for run-time process modelling – making use of wearables in BPM. In: Eibl M, Gaedke M (eds) Informatik. https://doi.org/10.18420/in2017_85
- Jannach D (2011) Recommender systems: an introduction. Cambridge University Press, New York
- Jannach D, Zanker M, Ge M, Gröning M (2012) Recommender systems in computer science and information systems – a landscape of research. In: Huemer C, Lops P (eds) E-commerce and web technologies. Springer, Heidelberg, pp 76–87
- Koschmider A (2007) Ähnlichkeitsbasierte Modellierungsunterstützung für Geschäftsprozesse. Universität Karlsruhe, Fakultät für Wirtschaftswissenschaften, PhD thesis. KITopen ID 1000007208
- Koschmider A, Hornung T, Oberweis A (2011) Recommendation-based editor for business process modeling. *Data Knowl Eng* 70(6):483–503. <https://doi.org/10.1016/j.datak.2011.02.002>
- Kuschke T, Mäder P (2014) Pattern-based auto-completion of UML modeling activities. In: Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering. ACM, New York, pp 551–556
- Lausen S (2002) Software requirements: styles and techniques. Pearson Education, London
- Lerchner H, Stary C (2016) Model while you work: towards effective and playful acquisition of stakeholder processes. In: Proceedings of the 8th International Conference on Subject-oriented Business Process Management. ACM, New York, pp 1–10
- Li Y, Cao B, Xu L et al (2014) An efficient recommendation method for improving business process modeling. *IEEE Trans Ind Inform* 10(1):502–513. <https://doi.org/10.1109/TII.2013.2258677>
- Luftman J, Zadeh HS, Derksen B et al (2012) Key information technology and management issues 2011–2012: an international

- study. *J Inf Technol* 27(3):198–212. <https://doi.org/10.1057/jit.2012.14>
- Luftman J, Zadeh HS, Derksen B et al (2013) Key information technology and management issues 2012–2013: an international study. *J Inf Technol* 28(4):354–366. <https://doi.org/10.1057/jit.2013.22>
- Matijacic M, Fellmann M, Özcan D, et al (2013) Elicitation and consolidation of requirements for mobile technical customer services support systems – a multi-method approach. In: *Proceedings of the 34th International Conference on Information Systems*. Milan
- Mazanek S, Minas M (2009) Business process models as a showcase for syntax-based assistance in diagram editors. In: Schürr A, Selic B (eds) *Model driven engineering languages and systems*. Springer, Heidelberg, pp 322–336
- Mazanek S, Maier S, Minas M (2008) Auto-completion for diagram editors based on graph grammars. In: *IEEE Symposium on visual languages and human-centric computing*, pp 242–245
- Melville P, Sindhvani V (2010) Recommender systems. In: Sammut C, Webb GI (eds) *Encyclopedia of machine learning*. Springer, New York, pp 829–838
- Metzger D, Niemöller C, Berkemeier L et al (2017) Vom Techniker zum Modellierer – Konzeption und Entwicklung eines Smart Glasses Systems zur Laufzeitmodellierung von Dienstleistungsprozessen. In: Thomas O, Nüttgens M, Fellmann M (eds) *Smart service engineering*. Springer Fachmedien, Wiesbaden, pp 193–213
- Nielen A, Költer D, Mütze-Niewöhner S, Karla J, Schlick CM (2011) An empirical analysis of human performance and error in process model development. In: Jeusfeld M, Delcambre L, Ling TW (eds) *Conceptual modeling – ER 2011. Lecture notes in computer science*, vol 6998. Springer, Heidelberg
- Niemöller C, Metzger D, Fellmann M, et al (2016) Shaping the future of mobile service support systems–ex-ante evaluation of smart glasses in technical customer service processes. In: Mayr, HC, Pinzger, M, (eds): *Informatik von Menschen für Menschen (Informatik 2016) (LNI 259)*, Klagenfurt, pp 753–770
- Niemöller C, Zobel B, Berkemeier L, et al (2017) Sind Smart Glasses die Zukunft der Digitalisierung von Arbeitsprozessen? Explorative Fallstudien zukünftiger Einsatzszenarien in der Logistik. In: Leimeister JM, Brenner W (eds): *Proceedings of the 13th International Conference on Wirtschaftsinformatik*, St. Gallen, pp 410–424
- Ricci F, Rokach L, Shapira B (2011) Introduction to recommender systems handbook. In: Ricci F, Rokach L, Shapira B, Kantor PB (eds) *recommender systems handbook*. Springer, Boston, pp 1–35
- Richardson C, Miers D (2013) *The Forrester Wave™: business process management suites*. Forrester Research
- Sarshar K, Weber M, Loos P (2006) Einsatz der Informationsmodellierung bei der Einführung betrieblicher Standardsoftware: Eine empirische Untersuchung bei Energieversorgerunternehmen. *Wirtschaftsinformatik* 48(2):120–127
- Sen S, Baudry B, Vangheluwe H (2010) Towards domain-specific model editors with automatic model completion. *Simul* 86(2):109–126. <https://doi.org/10.1177/0037549709340530>
- Shahzad K, Elias M, Johannesson P (2010) Requirements for a business process model repository: a stakeholders’ perspective. In: Abramowicz W, Tolksdorf R (eds) *Proceedings of the 13th international conference on business information systems*. Springer, Heidelberg, pp 158–170
- Starner T, Mann S, Rhodes B et al (1997) Augmented reality through wearable computing. *Presence Teleoper Virtual Environ* 6(4):386–398
- Thomas O (2005) Understanding the term reference model in information systems research: history, literature analysis and explanation. In: *Business Process Management Workshops*. Springer, Heidelberg, pp 484–496
- Thomas O (2006) Management von Referenzmodellen: Entwurf und Realisierung eines Informationssystems zur Entwicklung und Anwendung von Referenzmodellen. Logos, Berlin
- Thomas O, Fellmann M (2009) Semantic process modeling – design and implementation of an ontology-based representation of business processes. *Bus Inf Syst Eng* 1(6):438–451
- vom Brocke J, Simons A, Niehaves B, et al (2009) Reconstructing the giant: on the importance of rigour in documenting the literature search process. In: *Proceedings of the 17th European Conference on Information Systems*, Verona, pp 2206–2217
- Weber B, Reichert M, Rinderle-Ma S (2008) Change patterns and change support features – enhancing flexibility in process-aware information systems. *Data Knowl Eng* 66(3):438–466. <https://doi.org/10.1016/j.datak.2008.05.001>
- Webster J, Watson RT (2002) Analyzing the past to prepare for the future: writing a literature review. *MIS Q* 26(2):xiii–xxiii
- Wieloch K, Filipowska A, Kaczmarek M (2011) Autocompletion for business process modelling. In: Abramowicz W (ed) *Proceedings of the 14th international conference on business information systems*. Springer, Heidelberg, pp 30–40
- Wilmont I, Brinkkemper S, Weerd I, Hoppenbrouwers S (2010) Exploring intuitive modelling behaviour. In: Bider I, Halpin T, Krogstie J et al (eds) *Enterprise, business-process and information systems modeling: proceedings of the 11th international workshop, and 15th international conference, CAiSE 2010, Hammamet*. Springer, Heidelberg, pp 301–313
- zur Muehlen M, Recker J, Indulska M (2007) Sometimes less is more: are process modeling languages overly complex? In: *11th International IEEE EDOC Conference Workshop*, pp 197–204